

Cumplimiento ISO/IEC 27001:2022 — Xcapit FHE-ML Platform

Producto SaaS: Xcapit FHE-ML Platform **Version:** 1.0.0-rc.1 **Fecha:** 2026-03-02 **Preparado por:** Equipo de Seguridad, Xcapit **Clasificacion:** Confidencial

Resumen Ejecutivo

La Xcapit FHE-ML Platform es una plataforma SaaS de machine learning con privacidad integral, construida sobre Fully Homomorphic Encryption (FHE). Su propuesta de valor diferencial es permitir que multiples organizaciones ejecuten modelos de machine learning sobre datos cifrados sin que el servidor —ni ningun otro participante— pueda acceder a los datos en claro en ningun momento del proceso. Esto convierte a la plataforma en una solucion de “zero-knowledge computing” donde la privacidad esta garantizada por diseno criptografico, no por politicas de acceso.

El presente documento tiene como objetivo demostrar al auditor como la plataforma cumple con los 32 controles de la checklist de auditoria ISO/IEC 27001:2022 aplicables a un servicio SaaS desplegado en la nube. Para cada control se describe el contexto del requerimiento, la forma concreta en que la plataforma lo satisface, y la evidencia tecnica que lo respalda (fragmentos de configuracion, codigo fuente, arquitectura y procedimientos).

Stack tecnologico de la plataforma:

Capa	Tecnologia	Version	Funcion
Backend API	Django + Django REST Framework	5.2 LTS (soporte hasta abril 2028)	API REST v2, autenticacion, permisos, audit trail
Base de datos	PostgreSQL	16-alpine	Persistencia relacional, WAL para durabilidad
Cache y broker	Redis	7-alpine	Cache, sesiones, rate limiting, broker Celery
Procesamiento asincrono	Celery + Beat	Python 3.12	Tareas diferidas (blockchain, reportes, training)
Cifrado homomorfico	TenSEAL (esquema CKKS)	0.3.14	ML sobre datos cifrados (128/192/256-bit)
Blockchain	Arbitrum (Web3.py) + Solidity/Foundry	Web3.py 7.0.0	Gobernanza on-chain, registro de modelos, verificacion
Frontend	React 18 + Vite 5 + TailwindCSS 3	React 18.2	Dashboard SPA con internacionalizacion ES/EN
Gestion de secretos	OpenBao (fork open-source de Vault)	2.1.0	Almacenamiento seguro de claves y credenciales
Infraestructura	Docker multi-stage, AWS ECS Fargate	Python 3.12-slim	Contenedores hardened, non-root, read-only filesystem
CDN y hosting frontend	Vercel	-	Distribucion global, security headers, HTTPS

1. Requisitos del SGSI

4.1 Comprension de la organizacion y su contexto

Pregunta de auditoria: ¿El alcance del servicio SaaS esta documentado y definido dentro del SGSI?

Cumplimiento:

La plataforma ha sido disenada con una arquitectura modular clara donde cada componente tiene un alcance funcional bien definido y delimitado. Esta separacion permite identificar con precision que partes del sistema procesan que tipo de datos, facilitando tanto la evaluacion de riesgos como la aplicacion de controles especificos a cada modulo.

El sistema se compone de 7 aplicaciones Django independientes, cada una responsable de un dominio de negocio especifico:

Modulo	Descripcion funcional	Datos que gestiona
Core	Autenticacion de usuarios, sistema de permisos con 9 clases de autorizacion, registro de auditoria de todas las operaciones, gestion de API keys, webhooks de notificacion, health checks del sistema	Usuarios, empresas, tokens JWT, API keys (hasheadas), logs de auditoria
Consortiums	Creacion y gestion de consorcios multi-empresa, membresia con roles (owner/admin/member), contribucion de datos con pruebas criptograficas	Consortios, miembros, pruebas de contribucion
Governance	Propuestas de gobernanza, votacion ponderada por contribucion, ejecucion de decisiones, distribucion de recompensas, audit trail inmutable con hash chain	Propuestas, votos, distribuciones, eventos de auditoria encadenados
Compliance	Reglas de calidad de datos configurables por consorcio, alertas automaticas cuando los datos no cumplen umbrales	Reglas, resultados de validacion, alertas
Federated	Endpoints de inferencia sobre datos cifrados (FHE), gestion de nodos edge para procesamiento distribuido	Endpoints, requests de inferencia, resultados cifrados, nodos edge
Models	Registro y versionado de modelos de ML, ciclo de vida (draft → training → ready → deployed)	Modelos, versiones, metricas de entrenamiento
Marketplace	Catalogo de datasets y modelos disponibles para compartir entre empresas del consorcio	Listings, transacciones

Adicionalmente, la plataforma incluye:

- **SDK Python (v0.7.0):** Libreria cliente que permite a los usuarios cifrar datos localmente con FHE, entrenar modelos, y realizar inferencias. El cifrado ocurre en el lado del cliente, garantizando que los datos nunca salen del entorno del usuario en claro.
- **Dashboard React:** Panel de administracion SPA que consume exclusivamente la API REST v2. Implementa internacionalizacion en espanol e ingles.
- **Smart Contracts (Solidity/Foundry):** Tres contratos desplegados en Arbitrum para gobernanza on-chain, registro inmutable de modelos, y verificacion de computaciones.

Modelo de aislamiento multi-tenancy:

El aislamiento entre organizaciones es un pilar arquitectónico fundamental. Cada recurso del sistema está vinculado a una Company (empresa), y todo acceso a datos pasa por filtros automáticos que garantizan que una empresa nunca pueda ver, modificar o inferir datos de otra empresa:

1. **Filtrado automático en QuerySets:** Todos los ViewSets implementan `get_queryset()` filtrando por `request.user.company`, asegurando que las consultas a base de datos solo retornan datos de la empresa del usuario autenticado.
2. **9 clases de permisos:** El sistema de autorización verifica en cada request que el usuario tiene el rol adecuado (miembro de empresa, miembro de consorcio, owner, admin, etc.).
3. **Tests de aislamiento:** El suite de tests incluye fixtures específicos (`other_auth_client`) que simulan un usuario de otra empresa e intentan acceder a recursos ajenos, verificando que siempre reciben 0 resultados o errores 403/404.

Estado: Implementado

6.1.2 Evaluación de riesgos de seguridad de la información

Pregunta de auditoría: ¿Se ha realizado una evaluación de riesgos específica para el servicio SaaS en la nube?

Cumplimiento:

La plataforma cuenta con un análisis de riesgos formal basado en el framework **OWASP Top 10:2025**, que es el estándar de referencia de la industria para identificar las amenazas más críticas en aplicaciones web. Cada una de las 10 categorías OWASP fue evaluada contra la arquitectura de la plataforma, y para cada una se implementaron controles específicos que la mitigan. El resultado es que las 10 categorías se encuentran en estado **Mitigado**.

A continuación se detalla cada categoría con los controles implementados:

A01 — Broken Access Control (Control de acceso roto): La plataforma implementa un modelo de multi-tenancy estricto donde cada consulta a base de datos está automáticamente scoped a la empresa del usuario. Además, se cuenta con 9 clases de permisos que verifican roles a nivel de empresa, consorcio y recurso. El rate limiting por tiers previene abuso de endpoints.

A02 — Security Misconfiguration (Configuración insegura): Todos los security headers recomendados están habilitados (HSTS con 1 año de duración e includeSubDomains, X-Frame-Options DENY, X-Content-Type-Options nosniff, Content-Security-Policy restrictiva). CORS está configurado con whitelist explícita (sin wildcards). El modo DEBUG está deshabilitado en producción y se valida al arranque del servidor.

A03 — Supply Chain Failures (Fallos en la cadena de suministro): Las dependencias se escanean automáticamente en cada push con pip-audit (vulnerabilidades Python), Trivy y Gype (vulnerabilidades en contenedores).

Docker), CodeQL (análisis estático de código), y TruffleHog (detección de secretos filtrados). Dependabot genera PRs automáticos semanales para actualizar dependencias.

A04 — Cryptographic Failures (Fallos criptográficos): El diferenciador principal de la plataforma es FHE con esquema CKKS a 128, 192 o 256 bits de seguridad. Los JWT usan una clave de firma separada del SECRET_KEY de Django. Todos los secretos se gestionan vía OpenBao/Vault con TLS 1.2+.

A05 — Injection (Inyección): Se usa exclusivamente el ORM de Django para acceso a base de datos, eliminando el riesgo de SQL injection. Toda entrada de usuario pasa por serializadores de DRF con validación explícita. No se utiliza `eval()`, `exec()`, ni `subprocess` con input del usuario.

A06 — Insecure Design (Diseño inseguro): La arquitectura sigue el patrón Service Layer donde toda la lógica de negocio reside en servicios que retornan `ServiceResult[T]` (éxito o error tipado). Los errores siguen RFC 7807 (Problem Details). El audit trail registra todas las operaciones significativas.

A07 — Authentication Failures (Fallos de autenticación): Los JWT tienen rotación automática y blacklisting del token anterior en cada refresh. django-axes bloquea cuentas tras 5 intentos fallidos durante 30 minutos. Las passwords requieren mínimo 12 caracteres y pasan 4 validadores (similitud, longitud, passwords comunes, solo números).

A08 — Data Integrity Failures (Fallos de integridad): La gobernanza del consorcio se registra en una cadena de hashes SHA-256 (similar a blockchain) que hace detectable cualquier manipulación. Los contenedores Docker se escanean con dos herramientas independientes (Trivy + Gype). Todo merge requiere que los CI checks pasen.

A09 — Logging Failures (Fallos de logging): Logging estructurado en JSON con correlation IDs (UUID v4) que permiten trazar un request completo a través de todos los servicios. El modelo AuditLog registra actor, acción, recurso, IP, user-agent y timestamp de cada operación. Sentry captura errores en tiempo real en backend y frontend.

A10 — Exceptional Conditions (Condiciones excepcionales): Un exception handler custom convierte todas las excepciones no manejadas en respuestas RFC 7807 sin exponer detalles internos del servidor. El patrón ServiceResult envuelve todas las operaciones de servicio en un resultado tipado. Celery implementa retry con backoff exponencial para tareas fallidas.

Adicionalmente, se mantiene una **matriz de trazabilidad** que mapea cada historia de usuario a las categorías OWASP que mitiga, permitiendo verificar que ningún requisito de seguridad queda sin cobertura.

Estado: Implementado

2. Controles Organizacionales

A.5.1 Politicas para la seguridad de la informacion

Pregunta de auditoria: ¿Se ha aprobado y comunicado la politica de seguridad de la informacion?

Cumplimiento:

La organizacion mantiene una politica de seguridad formal, publicada y accesible para todos los miembros del equipo y contribuidores externos a traves del repositorio del proyecto. Esta politica se materializa en un documento de seguridad que cubre los siguientes aspectos:

Versiones soportadas con parches de seguridad:

La politica establece claramente que versiones del producto reciben actualizaciones de seguridad. Actualmente solo la version 1.0.x esta bajo soporte activo. Las versiones anteriores a 1.0 no reciben parches, incentivando a los usuarios a mantenerse en la version mas reciente.

Version	Soportada con parches de seguridad
1.0.x	Si
< 1.0	No

Proceso de reporte de vulnerabilidades:

Se ha establecido un canal dedicado de comunicacion segura para reportes de vulnerabilidades: **security@xcapit.com**. El proceso requiere que el reporter incluya informacion estructurada para facilitar la evaluacion y reproduccion del problema:

- Tipo de vulnerabilidad identificada (XSS, SQLi, IDOR, etc.)
- Archivos o componentes del codigo fuente afectados
- Pasos detallados para reproducir el problema
- Proof of Concept (PoC) funcional cuando sea posible
- Evaluacion del impacto potencial

Service Level Agreement (SLA) de respuesta:

La politica define tiempos de respuesta claros y medibles:

Fase	Tiempo maximo	Descripcion
Acuse de recibo	48 horas	Confirmacion de recepcion y asignacion de responsable
Status update	5 dias habiles	Evaluacion de severidad, plan de remediacion
Resolucion	90 dias	Target de correccion segun complejidad del issue
Divulgacion	Post-fix	Advisory publico con credito al reporter (salvo pedido de anonimato)

Guías de seguridad para usuarios de la plataforma:

La política documenta mejores prácticas que todo usuario del servicio debe seguir:

1. **API Keys:** Nunca commitear API keys a sistemas de control de versiones. Utilizar variables de entorno o gestores de secretos.
2. **Claves FHE:** Las claves de cifrado homomorfico deben almacenarse de forma segura en el entorno del cliente. Nunca deben aparecer en logs ni transmitirse en claro.
3. **Dependencias:** Mantener todas las dependencias del SDK actualizadas para recibir parches de seguridad.
4. **Control de acceso:** Aplicar el principio de mínimo privilegio al asignar permisos de API keys (usar `read` cuando no se necesita `write` o `admin`).

Guías de seguridad para contribuidores del código:

Todo desarrollador que contribuye al código de la plataforma debe adherir a las siguientes prácticas mandatorias:

1. Validar toda entrada de usuario a través de serializers con validación explícita.
2. Ejecutar `pip-audit` para verificar que no se introducen dependencias con vulnerabilidades conocidas.
3. Usar variables de entorno para toda información sensible (credenciales, claves, tokens). Nunca hardcodear secretos.
4. Todo Pull Request requiere revisión de seguridad por al menos un maintainer antes del merge.

Seguridad de smart contracts:

La política incluye requisitos específicos para el código Solidity desplegado en blockchain:

- **ReentrancyGuard** en todas las funciones que realizan llamadas externas, previniendo ataques de reentrancia.
- **Ownable2Step** para control de acceso con transferencia de propiedad en 2 pasos (propuesta + aceptación), evitando transferencias accidentales.
- Validación de entrada con errores custom descriptivos.

- Patron **pull-over-push** para distribucion de pagos, evitando que un destinatario malicioso bloquee la transaccion.

Estado: Implementado

A.5.2 Roles y responsabilidades en seguridad de la informacion

Pregunta de auditoria: ¿Se han asignado y documentado formalmente las funciones y responsabilidades del SGSI?

Cumplimiento:

La plataforma define roles de seguridad tanto a nivel organizacional como a nivel tecnico dentro del sistema, asegurando que cada responsabilidad tiene un propietario claro y que los permisos se otorgan segun el principio de minimo privilegio.

Roles organizacionales:

Rol	Responsabilidad	Como se ejerce
Security Lead	Punto de contacto para reportes de seguridad. Responsable de cumplir el SLA de respuesta (48h/5d/90d)	Gestiona el buzón security@xcapit.com, coordina remediacion
Code Reviewers	Revision obligatoria de todo cambio de codigo antes de su integracion	Minimo 1 maintainer debe aprobar cada Pull Request
CI/CD Pipeline	Ejecucion automatica de escaneos de seguridad en cada push	Job dedicado <code>security-scan</code> en GitHub Actions y GitLab CI

Roles dentro del sistema (implementados como clases de permisos):

El sistema de autorizacion implementa 9 clases de permisos que se aplican a nivel de cada endpoint de la API. Cada clase verifica un aspecto diferente de la identidad y el contexto del usuario:

```
class IsCompanyMember(BasePermission):
    """Verifica que el usuario autenticado pertenece a una empresa
    registrada.
    Este es el permiso base requerido por la mayoria de endpoints."""
    def has_permission(self, request, view):
        return request.user.is_authenticated and request.user.company
        is not None

class IsConsortiumMember(BasePermission):
    """Verifica que la empresa del usuario es miembro activo del
    consorcio solicitado.
    Implementa politica fail-closed: las operaciones de escritura sin
    contexto
    de consorcio son denegadas automaticamente. Para operaciones de
    lectura sin
    contexto explicito, delega al queryset scoping que ya filtra por
    empresa.
    Reconoce al owner del consorcio como miembro implicito."""
    def has_permission(self, request, view):
        consortium_id = self._extract_consortium_id(request, view)
        if not consortium_id and request.method not in SAFE_METHODS:
            return False # Fail-closed para escrituras sin contexto
        if Consortium.objects.filter(id=consortium_id,
            owner=request.user.company).exists():
            return True # Owner es miembro implicito
        return ConsortiumMember.objects.filter(
            consortium_id=consortium_id,
            company=request.user.company,
            status="active"
        ).exists()

class IsConsortiumOwner(BasePermission):
    """Solo el propietario (creador) del consorcio puede realizar
    esta accion.
    Usado para operaciones criticas como disolver el consorcio o
    cambiar configuracion."""

class IsConsortiumAdmin(BasePermission):
    """Propietarios o usuarios con rol 'admin' en el consorcio.
    Usado para operaciones de administracion como gestionar
    miembros."""

class HasAPIKeyPermission(BasePermission):
    """Verifica que la API key utilizada tiene el nivel de permiso
    requerido.
    Soporta 3 niveles: read (solo lectura), write (lectura y
    escritura),
    admin (acceso completo incluyendo gestion)."""

class IsResourceOwner(BasePermission):
    """Verifica que la empresa del usuario es la propietaria directa
    del recurso
    solicitado. Usado para operaciones de
    modificacion/eliminacion."""

class IsActiveUser(BasePermission):
    """Requiere que el usuario tenga estado activo (no suspendido ni
    eliminado)."""

class IsVerifiedCompany(BasePermission):
    """Requiere que la empresa del usuario haya completado el proceso
    de verificacion."""

class ReadOnly(BasePermission):
```

```
""""Restringe el acceso a metodos HTTP seguros (GET, HEAD,
OPTIONS).
Se combina con otros permisos para crear endpoints de solo
lectura."""
```

Politica de fail-closed:

Un aspecto critico del sistema de permisos es que opera con politica fail-closed: cuando una operacion de escritura (POST, PUT, PATCH, DELETE) se recibe sin contexto de consorcio suficiente para verificar la membresia, el permiso se deniega automaticamente. Esto previene que un error de programacion o una solicitud malformada pueda resultar en acceso no autorizado. Las operaciones de lectura sin contexto explicito delegan al queryset scoping, que de todas formas filtra los datos por la empresa del usuario.

Estado: Implementado

A.5.7 Inteligencia de amenazas

Pregunta de auditoria: ¿Se revisan fuentes de amenazas relevantes y se integran en la evaluacion de riesgos?

Cumplimiento:

La plataforma mantiene un sistema proactivo de deteccion de amenazas que opera de forma continua y automatizada, sin depender de revisiones manuales periodicas. Se ejecutan 6 herramientas de escaneo de seguridad especializadas, cada una enfocada en un vector de amenaza diferente, que en conjunto proporcionan cobertura integral de la superficie de ataque:

CodeQL (SAST — Static Application Security Testing): Analisis estatico del codigo fuente Python y JavaScript. Se ejecuta en cada push a las ramas principales, en cada Pull Request, y de forma programada los lunes a las 6 AM UTC. Utiliza los conjuntos de queries `security-extended` y `security-and-quality` de GitHub, que detectan inyeccion SQL, XSS, deserializacion insegura, uso de criptografia debil, y otros patrones de vulnerabilidad. Los resultados se publican como alertas SARIF en la pestana Security de GitHub.

pip-audit (vulnerabilidades en dependencias Python): Audita en cada push todas las dependencias contra la base de datos de vulnerabilidades PyPI Advisory Database. Si detecta una dependencia con CVE conocida, el build falla y el equipo recibe notificacion inmediata.

safety (scanner secundario de dependencias Python): Complementa pip-audit consultando la base de datos Safety DB como fuente adicional. La redundancia de dos scanners reduce el riesgo de que una vulnerabilidad pase desapercibida por depender de una sola fuente.

TruffleHog v3.88.0 (deteccion de secretos en commits): Escanea el historial de commits buscando API keys, passwords, tokens y otros secretos filtrados accidentalmente. Opera con `--only-verified`, reportando solo secretos que puede confirmar como validos, reduciendo falsos positivos.

Trivy (vulnerabilidades en contenedores Docker): Escanea la imagen Docker completa buscando CVEs de severidad CRITICAL y HIGH. Configurado con `exit-code: 1`, **bloquea el build** si encuentra vulnerabilidades críticas. Los resultados se cargan como SARIF al tab de seguridad de GitHub.

Grype (scanner secundario de contenedores): Segunda opinion sobre vulnerabilidades en contenedores, consultando la base de datos de Anchore. Tambien carga resultados como SARIF para correlacion cruzada.

Actualizaciones automaticas de dependencias (Dependabot):

Ecosistema	Frecuencia	Limite de PRs simultaneos
Python (pip)	Semanal	5 PRs abiertas maximo
JavaScript (npm)	Semanal	Dashboard y SDK TypeScript
GitHub Actions	Semanal	Workflows actualizados
Docker base images	Mensual	Imagenes base actualizadas

Estado: Implementado

A.5.9 Inventario de informacion y otros activos asociados

Pregunta de auditoria: ¿Existe un inventario completo de activos cloud, aplicaciones, datos y APIs? ¿Se ha asignado un propietario a cada activo?

Cumplimiento:

La plataforma mantiene un inventario detallado de todos sus componentes de infraestructura, dependencias de software y activos desplegados. Cada componente tiene una funcion definida y su version esta controlada.

Servicios de infraestructura:

Servicio	Version	Funcion	Persistencia
PostgreSQL	16-alpine	Base de datos relacional principal	Volume postgres_data + WAL
Redis	7-alpine	Cache, broker Celery, sesiones, rate limiting	Volume redis_data + AOF
OpenBao	2.1.0	Gestion centralizada de secretos	Backend PostgreSQL con HA
Django API	Python 3.12, Gunicorn	Backend REST API v2	Stateless
Celery Worker	Python 3.12	Procesamiento asincrono (blockchain, reportes, training)	Stateless
Celery Beat	Python 3.12	Tareas programadas periodicas	Stateless

Dependencias Python del backend (64 paquetes):

Categoria	Paquetes principales	Funcion
Framework	Django 5.2 LTS, DRF 3.15+	Framework web y API REST
Autenticacion	simplejwt 5.3.1, django-axes 6.5.0, django-ratelimit 4.1.0	JWT, anti brute-force, rate limiting
Cifrado	TenSEAL 0.3.14, django-encrypted-model-fields 0.6.5	FHE homomorfo, Fernet at-rest
Blockchain	Web3.py 7.0.0, eth-account 0.13.0	Smart contracts Arbitrum
Secretos	hvac 2.3.0	Cliente OpenBao/Vault
Monitoreo	sentry-sdk 2.0.0, python-json-logger 2.0.0, django-health-check 3.18.0	Errores, logging, health
ML	scikit-learn 1.5.0, numpy 1.24.0	Modelos compatibles con FHE

Frontend: React 18.2, Vite 5.0, TailwindCSS 3.3.6, Sentry React 10.38.0, i18next.

Smart contracts: OpenZeppelin v5.6.1, forge-std v1.15.0 (lockfile con hashes de integridad).

Contratos desplegados en Arbitrum: Governance Contract, Model Registry, Computation Verifier — en Arbitrum Sepolia (testnet) y Arbitrum One (mainnet).

Estado: Implementado

A.5.15 Control de acceso

Pregunta de auditoria: ¿Se ha definido e implementado la politica de control de acceso? ¿MFA habilitado? ¿Gestion de cuentas privilegiadas?

Cumplimiento:

El control de acceso se implementa en multiples capas complementarias e independientes (defensa en profundidad), abarcando autenticacion, autorizacion, proteccion contra fuerza bruta, y gestion del ciclo de vida de credenciales.

Autenticacion JWT con rotacion y blacklisting:

```
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=30),    # Token de
    # acceso expira en 30 minutos
    "REFRESH_TOKEN_LIFETIME": timedelta(days=1),      # Refresh
    # expira en 1 dia
    "ROTATE_REFRESH_TOKENS": True,                    # Cada refresh
    # emite un token nuevo
    "BLACKLIST_AFTER_ROTATION": True,                 # El refresh
    # anterior se invalida
    "UPDATE_LAST_LOGIN": True,                        # Registra
    # fecha del ultimo login
    "ALGORITHM": "HS256",
    "SIGNING_KEY": JWT_SIGNING_KEY,                  # Clave
    # separada del SECRET_KEY
    "TOKEN_BLACKLIST_ENABLED": True,                  # Revocacion
    # inmediata habilitada
}
```

La clave de firma JWT es independiente del SECRET_KEY de Django. Al cambiar password, todos los refresh tokens se invalidan automaticamente. El endpoint de logout blacklista el refresh token de inmediato.

Autenticacion por API Key:

Para integraciones programaticas, las API keys ofrecen: generacion con `secrets.token_urlsafe(32)` (256 bits de entropia), almacenamiento como hash SHA-256 (nunca en plaintext), prefijo publico de 8 caracteres para identificacion en logs, expiracion configurable, tracking de ultimo uso, permisos granulares (read/write/admin), y rate limiting individual por key.

Proteccion contra fuerza bruta:

```
AXES_ENABLED = True
AXES_FAILURE_LIMIT = 5 # Max 5
    intentos fallidos
AXES_COOLOFF_TIME = timedelta(minutes=30) # Lockout 30
    minutos
AXES_LOCK_OUT_BY_COMBINATION_USER_AND_IP = True # Por usuario
    + IP
AXES_RESET_ON_SUCCESS = True # Reset al
    login exitoso
```

Politica de passwords (4 validadores simultaneos):

```
AUTH_PASSWORD_VALIDATORS = [
    {"NAME": "UserAttributeSimilarityValidator"}, # No similar
        a datos del usuario
    {"NAME": "MinimumLengthValidator",
     "OPTIONS": {"min_length": 12}}, # Minimo 12
        caracteres
    {"NAME": "CommonPasswordValidator"}, # No
        passwords comunes (20,000 lista)
    {"NAME": "NumericPasswordValidator"}, # No solo
        numeros
]
```

Estado: Implementado

A.5.19 Seguridad de la informacion en las relaciones con proveedores

Pregunta de auditoria: ¿Los contratos con proveedores incluyen clausulas de confidencialidad, disponibilidad e incidentes?

Cumplimiento:

La plataforma minimiza la dependencia de proveedores propietarios y gestiona activamente el riesgo de la cadena de suministro de software. Toda dependencia de terceros esta sujeta a controles automatizados:

Control	Implementacion	Proposito
Versiones pinned	Version minima en <code>requirements.txt</code> y <code>package.json</code>	Previene actualizaciones involuntarias
Lockfiles con hashes	<code>package-lock.json</code> con SHA-512	Garantiza integridad del paquete
Escaneo en CI	pip-audit + safety en cada push	Detecta CVEs antes del merge
Container scanning	Trivy + Gype	Vulnerabilidades en imagen Docker completa
Updates automaticos	Dependabot semanal/mensual	Mantiene dependencias actualizadas
Deteccion de secretos	TruffleHog en cada commit	Previene filtracion de credenciales
SAST	CodeQL Python + JavaScript	Detecta uso inseguro de librerias

Portabilidad (mitigacion de vendor lock-in): PostgreSQL (SQL estandar), Redis (protocolo estandar, compatible con Valkey/KeyDB), Docker (estandar OCI), OpenBao (fork open-source de Vault), Arbitrum (EVM-compatible, portable a Ethereum/Polygon/Optimism).

Estado: Implementado

A.5.22 Seguimiento, revision y gestion del cambio de proveedores

Pregunta de auditoria: ¿Se monitorea el desempeno de los proveedores?
¿Revisiones periodicas de SLA?

Cumplimiento:

El monitoreo de componentes y servicios es continuo y automatizado:

Control	Implementacion
Health checks	PostgreSQL (10s), Redis (10s), Django (/health/ 30s), OpenBao (10s) con umbrales de degradacion
Dependabot	PRs automaticos semanales/mensuales con changelog de cambios
CI dual	GitHub Actions (10 jobs) + GitLab CI (9 jobs) — redundancia de plataforma
Usage tracking	Modelo <code>UsageTracking</code> : requests, predicciones, rate_limit_hits por empresa/dia
Container scanning	Base images actualizadas mensualmente via Dependabot
Metricas de latencia	Middleware registra <code>duration_ms</code> en cada request

Estado: Implementado

A.5.23 Seguridad de la informacion para el uso de servicios en la nube

Pregunta de auditoria: ¿Se ha evaluado el riesgo del proveedor cloud?
¿Modelo de responsabilidad compartida documentado?

Cumplimiento:

La plataforma opera en un modelo de responsabilidad compartida: Xcapit gestiona la seguridad de aplicacion, datos y configuracion; los proveedores (AWS, Vercel) gestionan la infraestructura fisica y de red.

Vercel (frontend): Security headers configurados explicitamente — CSP restrictivo (`default-src 'self'` ; solo conecta con `apifhe.xcapit.com`), HSTS 2 anos con preload, Permissions-Policy deshabilitando camara/microfono/geolocalizacion/pagos, X-Frame-Options DENY.

AWS ECS Fargate (backend): Network mode awsvpc (ENI dedicado por tarea), secretos via AWS Secrets Manager, IAM roles con minimo privilegio, CloudWatch Logs con retencion configurable, health checks cada 30 segundos.

Hardening de contenedores: Usuario non-root (UID 1000), filesystem read-only (`read_only: true`), solo `/tmp` writable en RAM, multi-stage build (sin herramientas dev en produccion), base image `python:3.12-slim` , archivos estaticos montados read-only.

Estado: Implementado

A.5.24 Planificacion y preparacion de la gestion de incidentes

Pregunta de auditoria: ¿Existe un proceso formal de respuesta a incidentes?
¿Registro de incidentes? ¿Lecciones aprendidas?

Cumplimiento:

Proceso de respuesta:

1. Reporter envia email a **security@xcapit.com** con tipo, componentes afectados, pasos de reproduccion, PoC, impacto.
2. **48 horas:** Acuse de recibo y asignacion de responsable.
3. **5 dias habiles:** Evaluacion de severidad y plan de remediacion.
4. **90 dias:** Target de correccion segun complejidad.
5. **Post-fix:** Advisory publico con credito al reporter.

Deteccion automatizada (Sentry):

```
sentry_sdk.init(  
    dsn=SENTRY_DSN,  
    integrations=[DjangoIntegration(), CeleryIntegration(),  
                  RedisIntegration()],  
    traces_sample_rate=0.25,      # 25% muestreo de rendimiento  
    profiles_sample_rate=0.1,     # 10% profiling  
    send_default_pii=False,      # NUNCA envia PII a Sentry  
    environment="production",  
)
```

Frontend: Sentry React SDK v10.38.0 para error tracking en dashboard.

Estado: Implementado

A.5.25 Evaluacion y decision sobre eventos de seguridad

Pregunta de auditoria: ¿Existe procedimiento documentado para la gestion de eventos de seguridad?

Cumplimiento:

El sistema registra eventos de seguridad en dos niveles complementarios.

Nivel 1 — AuditLog operacional:

```
class AuditLog(models.Model):  
    id = models.UUIDField(primary_key=True, default=uuid.uuid4)  
    user = models.ForeignKey(User, null=True,  
                             on_delete=models.SET_NULL)  
    company = models.ForeignKey(Company, null=True,  
                                on_delete=models.SET_NULL)  
    api_key_name = models.CharField(max_length=255, blank=True)  
    action = models.CharField(max_length=255, db_index=True)  
    resource_type = models.CharField(max_length=100, db_index=True)  
    resource_id = models.CharField(max_length=255, blank=True)  
    ip_address = models.GenericIPAddressField(null=True)  
    user_agent = models.CharField(max_length=500, blank=True)  
    request_path = models.CharField(max_length=500, blank=True)  
    request_method = models.CharField(max_length=10, blank=True)  
    response_status = models.IntegerField(null=True)  
    extra_data = models.JSONField(default=dict, blank=True)  
    created_at = models.DateTimeField(auto_now_add=True,  
                                     db_index=True)
```

Eventos registrados: `user_registered` , `user_logged_in` , `user_logged_out` , `password_changed` , `api_key_created` , `endpoint_created` , `proposal_created/voted/executed` .

Correlation IDs: Cada request recibe UUID v4 unico (generado o extraido de headers `X-Correlation-ID` , `X-Request-ID` , `X-Trace-ID`), inyectado en todos los logs y retornado en la respuesta. Permite trazar un request completo a traves de todos los servicios.

Redaccion de datos sensibles: Headers `HTTP_AUTHORIZATION` , `HTTP_X_API_KEY` , `HTTP_COOKIE` redactados. Query params `password` , `token` , `key` , `secret` excluidos de logs.

Nivel 2 — Audit trail de gobernanza con hash chain immutable:

Los eventos de gobernanza usan un modelo `AuditEvent` con hash SHA-256 encadenado (`event_hash` + `previous_hash`). Cada evento calcula su hash a partir de

`consortium_id:event_type:actor_id:target_id:previous_hash:created_at` .

Cubre 14 tipos de eventos (creacion de consorcio, union/salida de miembros, contribuciones, propuestas, votos, recompensas, configuracion). El endpoint `GET /api/v2/governance/audit-events/verify/` valida la integridad completa de la cadena.

Estado: Implementado

A.5.30 Preparacion para las TIC para la continuidad del negocio

Pregunta de auditoria: ¿Existe BIA? ¿Se han definido y probado RTO/RPO? ¿Test de DR documentado?

Cumplimiento:

La plataforma implementa multiples mecanismos de continuidad que cubren desde la deteccion temprana de degradacion hasta la recuperacion automatica de servicios caidos.

Health checks en 3 niveles (Kubernetes-compatible):

Endpoint	Funcion	Comportamiento
<code>/health/</code>	Health check completo	Verifica DB + Redis + blockchain RPC. Retorna status por componente con latencia. HTTP 200 si saludable/degradado, 503 si no saludable
Liveness	Proceso vivo	Retorna 200 si el proceso responde (independiente de dependencias)
Readiness	Dependencias listas	Retorna 503 si DB o Redis no disponibles, previniendo que el balanceador envíe trafico

Umbral de degradación: Database > 1 segundo = DEGRADED. Redis > 100ms = DEGRADED. Blockchain RPC timeout 5 segundos (tratado como no crítico, solo alerta).

Políticas de restart automático:

```
deploy:
  restart_policy:
    condition: on-failure      # Solo reinicia si el servicio falla
    delay: 5s                  # Espera 5 segundos entre reintentos
    max_attempts: 3            # Maximo 3 intentos
    window: 120s               # Ventana de evaluacion de 2 minutos
```

Persistencia de datos:

Servicio	Mecanismo	Configuración
PostgreSQL	WAL (Write-Ahead Log)	<code>max_wal_size=4GB</code> , <code>min_wal_size=1GB</code> , checkpoint target 0.9
Redis	AOF (Append Only File)	<code>appendonly yes</code> , <code>appendfsync everysec</code> , <code>maxmemory 256mb</code>
OpenBao	PostgreSQL backend	HA con distributed locks

Worker recycling (prevención de memory leaks):

```
# gunicorn.conf.py
workers = int(os.environ.get("GUNICORN_WORKERS", "4"))
threads = int(os.environ.get("GUNICORN_THREADS", "2"))
timeout = int(os.environ.get("GUNICORN_TIMEOUT", "120"))
graceful_timeout = 10
max_requests = int(os.environ.get("GUNICORN_MAX_REQUESTS", "1000"))
max_requests_jitter = 50      # Previene reinicios sincronizados entre  
workers
```

Resource limits de producción:

Servicio	CPU	Memoria
Django	2.0	2 GB
PostgreSQL	1.0	1 GB
Celery	1.0	1 GB
Redis	0.5	512 MB
Celery Beat	0.25	256 MB

Dependencias de arranque: Django no inicia hasta que PostgreSQL y Redis estén healthy (health check dependency en Docker Compose).

Estado: Implementado

A.5.31 Identificación de requisitos legales, reglamentarios y contractuales

Pregunta de auditoria: ¿Se han identificado y revisado los requisitos legales aplicables?

Cumplimiento:

La plataforma ha sido diseñada para cumplir con los requisitos regulatorios mas exigentes en materia de privacidad de datos, gracias a su arquitectura de zero-knowledge computing basada en FHE:

Requisito	Como la plataforma lo cumple
GDPR Art. 25 (Privacy by Design)	El servidor nunca accede a datos en claro. FHE permite procesamiento de ML sobre datos cifrados, cumpliendo privacy by design de forma criptografica, no solo por politica
HIPAA (datos de salud)	Los datos de salud se procesan cifrados end-to-end via FHE. Ni la plataforma ni los operadores del consorcio pueden descifrar los datos de los participantes
AGPL-3.0 (licencia)	El proyecto es open-source bajo licencia AGPL-3.0, documentada en el repositorio
PII handling	<code>send_default_pii=False</code> en Sentry, headers sensibles redactados en logs, query params con passwords/tokens excluidos
Divulgacion coordinada	Politica documentada con SLA: 48h acuse, 5d evaluacion, 90d resolucion

Estado: Implementado

A.5.33 Revision de seguridad de la informacion

Pregunta de auditoria: ¿Existen metricas e indicadores de seguridad para el servicio SaaS?

Cumplimiento:

La plataforma mantiene las siguientes metricas de seguridad medibles y verificables:

Metrica / KPI	Valor actual	Observacion
Test coverage	96.23%	Umbral CI: 90%. Todo PR que baje la cobertura es rechazado
Total tests	2,116	1,496 Django + 620 SDK
CI/CD pipelines	10 + 9 jobs	GitHub Actions + GitLab CI (redundancia)
Security scanners	6	CodeQL, pip-audit, safety, TruffleHog, Trivy, Gype
Pre-commit hooks	11	Formatting, linting, secretos, tests
OWASP Top 10	10/10 mitigadas	Cada categoria con controles documentados
Security headers	7	HSTS, X-Frame-Options, nosniff, CSP, Referrer, XSS, COOP
Permission classes	9	Autorizacion granular por endpoint
Middleware layers	13	Stack de seguridad en capas

Tracking de uso por empresa:

```
class UsageTracking(models.Model):
    company = models.ForeignKey(Company, on_delete=models.CASCADE)
    date = models.DateField()
    request_count = models.IntegerField(default=0)
    prediction_count = models.IntegerField(default=0)
    training_count = models.IntegerField(default=0)
    data_upload_bytes = models.BigIntegerField(default=0)
    rate_limit_hits = models.IntegerField(default=0)
    # Constraint unico por company + fecha
```

Estado: Implementado

A.5.34 Privacidad y proteccion de datos de caracter personal (DCP)

Pregunta de auditoria: ¿Se han implementado medidas de proteccion de la informacion de identificacion personal?

Cumplimiento:

La privacidad es el diferenciador core de la plataforma. A diferencia de sistemas convencionales que dependen de políticas de acceso para proteger datos, Xcapit FHE-ML Platform implementa protección criptográfica mediante Fully Homomorphic Encryption (esquema CKKS via TenSEAL):

Flujo de datos con FHE (zero-knowledge):

1. **El cliente cifra sus datos localmente** usando su clave privada con seguridad de 128, 192 o 256 bits (configurable, validado al arranque del servidor).
2. **El servidor recibe y procesa datos cifrados** — ejecuta inferencia de ML directamente sobre el ciphertext sin poder descifrarlos.
3. **El resultado cifrado retorna al cliente** — sigue siendo ciphertext indescifrable para el servidor.
4. **Solo el cliente descifra el resultado** con su clave privada local.

El servidor **nunca ve los datos en claro**. Incluso un breach completo del servidor no expone datos de clientes, porque el servidor no posee las claves de descifrado.

```
# Validacion de nivel de seguridad FHE al arranque
FHE_SECURITY_LEVEL = int(os.environ.get("FHE_SECURITY_LEVEL", "128"))
if FHE_SECURITY_LEVEL not in [128, 192, 256]:
    raise ValueError("FHE_SECURITY_LEVEL must be 128, 192, or 256")
```

Modelos ML soportados sobre datos cifrados: LinearRegression, LogisticRegression, DecisionTree, KMeans — todos ejecutables sobre ciphertext sin descifrar.

Controles adicionales de privacidad:

Control	Implementacion
Sentry PII	<code>send_default_pii=False</code> — no envia emails, tokens ni IPs a Sentry
Headers redactados	<code>HTTP_AUTHORIZATION</code> , <code>HTTP_X_API_KEY</code> , <code>HTTP_COOKIE</code> excluidos de logs
Query params	<code>password</code> , <code>token</code> , <code>key</code> , <code>secret</code> excluidos de request logging
Field encryption	Webhook secrets cifrados con Fernet (EncryptedCharField) en base de datos
HTTPS obligatorio	<code>SECURE_SSL_REDIRECT=True</code> en produccion
Cookies	<code>Secure=True</code> , <code>HttpOnly=True</code> , <code>SameSite=Lax</code>
CSP frontend	<code>default-src 'self'</code> — sin trackers ni analytics externos

Estado: Implementado

3. Controles Tecnologicos — Infraestructura

A.8.2 Gestion de privilegios de acceso

Pregunta de auditoria: ¿Se controlan y revisan periodicamente las cuentas privilegiadas? ¿MFA y control de roles?

Cumplimiento:

El sistema implementa una jerarquia de roles por consorcio con permisos crecientes, combinada con rate limiting multi-tier que adapta los limites de uso segun el plan de la empresa:

Jerarquia de roles por consorcio:

Rol	Permisos	Validacion
Owner	Control total del consorcio, gestion de miembros, disolucion	Verificado por <code>IsConsortiumOwner</code>
Admin	Administracion, moderacion, gestion de miembros	Verificado por <code>IsConsortiumAdmin</code>
Member	Lectura, votacion, contribucion de datos	Verificado por <code>IsConsortiumMember</code>
API Key	Granular: read / write / admin	Verificado por <code>HasAPIKeyPermission</code>

Rate limiting por tier de empresa:

Tier	Requests/minuto	Requests/dia	Modelos ML	Consorcios
Free	10	100	2	1
Starter	100	5,000	10	5
Professional	500	50,000	50	20
Enterprise	2,000	Ilimitado	Ilimitado	Ilimitado

El rate limiting se implementa con Redis como backend y sliding window de 60 segundos. Las respuestas incluyen headers `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`, y `Retry-After` en caso de 429.

API Key tracking: `last_used_at` actualizado en cada uso, `expires_at` validado automaticamente, prefijo publico de 8 chars para identificacion sin exponer la clave, hash SHA-256 en base de datos (clave original nunca almacenada).

Estado: Implementado

A.8.8 Gestion de vulnerabilidades tecnicas

Pregunta de auditoria: ¿Se realizan analisis de vulnerabilidades y se corrigen?
¿Es periodico?

Cumplimiento:

La plataforma ejecuta un pipeline de seguridad automatizado en cada push y Pull Request, asegurando que ningun codigo con vulnerabilidades conocidas llega a produccion:

```
push/PR → lint (ruff) → tests (coverage >=90%) → security-scan →  
container-scan → SARIF → GitHub Security
```

Detalle de cada scanner:

- 1. CodeQL (SAST):** Analisis estatico de Python y JavaScript con queries `security-extended` y `security-and-quality`. Se ejecuta en push, PRs, y cron semanal los lunes a las 6 AM.
 - 2. pip-audit + safety:** Auditan todas las dependencias Python contra bases de datos de CVEs en cada push.
 - 3. TruffleHog v3.88.0:** Deteccion de secretos en commits con `--only-verified` para minimizar falsos positivos.
 - 4. Trivy (contenedores):** Escanea imagen Docker por CVEs CRITICAL/HIGH. `exit-code: 1` bloquea el build. Resultados SARIF en GitHub Security.
 - 5. Gype (contenedores, scanner secundario):** Validacion cruzada con base de datos Anchore. Resultados SARIF.
- Pre-commit hooks de seguridad:** `detect-private-key` bloquea commits con claves privadas. `check-added-large-files` (max 1000KB) previene binarios accidentales.

Pipeline dual: Los mismos escaneos corren en GitHub Actions (10 jobs) Y GitLab CI (9 jobs) para redundancia.

Estado: Implementado

A.8.9 Gestion de la configuracion

Pregunta de auditoria: ¿Se define y mantiene la configuracion basica de la nube? ¿Hardening documentado?

Cumplimiento:

La plataforma implementa validacion estricta de configuracion al arranque, rechazando iniciar si falta cualquier configuracion critica de seguridad:

```
# settings.py - Falla inmediatamente si falta configuracion critica
SECRET_KEY = os.environ.get("DJANGO_SECRET_KEY")
if not SECRET_KEY:
    raise ValueError("DJANGO_SECRET_KEY environment variable is
        required")

FIELD_ENCRYPTION_KEY = os.environ.get("FIELD_ENCRYPTION_KEY")
if not FIELD_ENCRYPTION_KEY and not DEBUG:
    raise ValueError("FIELD_ENCRYPTION_KEY environment variable is
        required in production")

# Rechaza SQLite en produccion
if not DEBUG and "sqlite" in DATABASES["default"]["ENGINE"]:
    raise ValueError("SQLite is not allowed in production")

# Valida nivel de seguridad FHE
FHE_SECURITY_LEVEL = int(os.environ.get("FHE_SECURITY_LEVEL", "128"))
if FHE_SECURITY_LEVEL not in [128, 192, 256]:
    raise ValueError("FHE_SECURITY_LEVEL must be 128, 192, or 256")
```

Security headers (triple capa: Django + Nginx + Vercel):

Header	Valor	Donde se aplica
Strict-Transport-Security	max-age=31536000; includeSubDomains; preload	Django + Nginx + Vercel
X-Frame-Options	DENY	Django + Nginx
X-Content-Type-Options	nosniff	Django + Nginx + Vercel
X-XSS-Protection	1; mode=block	Django + Nginx
Referrer-Policy	strict-origin-when-cross-origin	Django + Nginx + Vercel
Content-Security-Policy	default-src 'none'; frame-ancestors 'none' (API) / default-src 'self' (frontend)	Nginx + Vercel
Permissions-Policy	camera=(), microphone=(), geolocation=(), payment=()	Vercel

Hardening de contenedores: Non-root user (UID 1000), read-only filesystem, multi-stage build, base image `python:3.12-slim`, sin herramientas de desarrollo en imagen de produccion.

Redis con password en produccion:

```
command: redis-server --requirepass ${REDIS_PASSWORD:?REDIS_PASSWORD
    is required}
```

Network isolation (Docker): Network bridge dedicada. PostgreSQL, Redis, OpenBao sin ports expuestos en produccion. Solo el reverse proxy expone el API.

OpenBao/Vault hardening: TLS 1.2+ con cipher suites modernos (AES-256-GCM-SHA384), PostgreSQL backend con SSL required, memory locking (secretos no se swapean a disco), politicas read-only para aplicaciones.

Estado: Implementado

A.8.13 Copias de seguridad de la informacion

Pregunta de auditoria: ¿Se realizan copias de seguridad y se comprueba la restauracion?

Cumplimiento:

Servicio	Mecanismo de persistencia	Configuracion
PostgreSQL	Volume nombrado <code>postgres_data</code> + WAL	<code>max_wal_size=4GB</code> , <code>min_wal_size=1GB</code> , checkpoint 0.9, <code>shared_buffers=256MB</code>
Redis	Volume nombrado <code>redis_data</code> + AOF	<code>appendonly yes</code> , <code>appendfsync</code> <code>everysec</code> , <code>maxmemory</code> <code>256mb</code>
OpenBao	PostgreSQL backend con HA	Tables: <code>vault_kv_store</code> , <code>vault_ha_locks</code>
Logs	JSON file driver con rotacion	<code>max-size: 50m</code> , <code>max-file: 5</code>

Estado: Implementado

A.8.15 Registros de eventos

Pregunta de auditoria: ¿La plataforma realiza registro y monitoreo de eventos de seguridad? ¿Logs conservados adecuadamente?

Cumplimiento:

Structured JSON logging: Cada entry contiene timestamp, level, logger, service name, environment, correlation_id, user_id, company_id, request path/method, duration_ms, y status_code.

Middleware stack de seguridad (13 capas, orden critico):

```
MIDDLEWARE = [  
    "CorrelationIdMiddleware",      # 1. Trazabilidad (UUID v4  
        por request)  
    "SecurityMiddleware",          # 2. HSTS, XSS filter,  
        nosniff  
    "CorsMiddleware",             # 3. CORS whitelist  
    "WhiteNoiseMiddleware",       # 4. Static files  
    "SessionMiddleware",          # 5. Sesiones  
    "CommonMiddleware",           # 6. URL normalization  
    "CsrfViewMiddleware",         # 7. CSRF tokens  
    "AuthenticationMiddleware",   # 8. Autenticacion  
    "MessageMiddleware",          # 9. Messages framework  
    "XFrameOptionsMiddleware",    # 10. Clickjacking protection  
    "AxesMiddleware",             # 11. Brute-force (despues de  
        auth)  
    "RateLimitMiddleware",        # 12. Rate limiting (despues  
        de auth)  
    "RequestLoggingMiddleware",   # 13. Logging (ultimo,  
        captura user info)  
]
```

Log levels por codigo HTTP: ERROR para ≥ 500 , WARNING para 400-499, INFO para < 400 .

Retencion: JSON file driver con rotacion 50MB x 5 archivos. En AWS: CloudWatch Logs con retencion configurable.

Estado: Implementado

A.8.16 Seguimiento de actividades

Pregunta de auditoria: ¿Se supervisan y revisan los registros? ¿Alertas activas?

Cumplimiento:

Control	Implementacion
Error tracking real-time	Sentry con integraciones Django + Celery + Redis
Frontend errors	Sentry React SDK en dashboard
Rate limit alerts	Webhook automatico al exceder limite (<code>security.ratelimit.exceeded</code>)
Health degradation	DB > 1s o Redis > 100ms = DEGRADED
Webhook delivery tracking	Modelo <code>WebhookDelivery</code> : status, intentos, errores, latencia
Governance verification	<code>GET /verify/</code> valida integridad de hash chain
Usage tracking	<code>UsageTracking</code> : requests, predicciones, rate_limit_hits por empresa/dia

Estado: Implementado

A.8.20 Seguridad de redes

Pregunta de auditoria: ¿Se configura la seguridad de red (WAF, segmentacion, cortafuegos)?

Cumplimiento:

CORS (whitelist estricta, sin wildcards):

```
CORS_ALLOWED_ORIGINS = [  
    "https://xcapit-privacy.vercel.app",  
    "https://appfhe.xcapit.com",  
    "https://privacy.xcapit.com",  
]  
CORS_ALLOW_CREDENTIALS = True
```

CSRF: `CSRF_TRUSTED_ORIGINS` con whitelist, `CSRF_COOKIE_SECURE=True`, `CSRF_COOKIE_HTTPONLY=True`.

Proteccion SSRF:

```
@staticmethod  
def _is_internal_url(url):  
    """Bloquea URLs internas para prevenir SSRF."""  
    hostname = urlparse(url).hostname  
    if hostname in ("localhost", "127.0.0.1", "0.0.0.0", "::1"):  
        return True  
    # Resolucion DNS + verificacion de IP  
    for result in socket.getaddrinfo(hostname, None):  
        ip = ipaddress.ip_address(result[4][0])  
        if ip.is_private or ip.is_loopback or ip.is_link_local:  
            return True  
    return False
```

Rate limiting: Anonimos 100/hora, autenticados 1000/hora, + tier-based per-company con Redis.

Network isolation (Docker): Bridge network dedicada, servicios internos sin ports expuestos en produccion.

Estado: Implementado

A.8.24 Uso de la criptografia

Pregunta de auditoria: ¿Se aplica cifrado en transito y en reposo? ¿Gestion de claves documentada?

Cumplimiento:

Cifrado en transito:

Control	Configuracion
HSTS	31536000s (1 ano), includeSubDomains, preload
SSL redirect	Forzado en produccion
Cookies Secure	CSRF + Session solo HTTPS
OpenBao TLS	TLS 1.2+ con AES-256-GCM-SHA384

Cifrado en reposo:

Dato	Mecanismo	Detalle
Datos ML (cliente)	FHE CKKS	128/192/256-bit, procesados sin descifrar
Webhook secrets	Fernet (EncryptedCharField)	<code>FIELD_ENCRYPTION_KEY</code> requerida
API keys	SHA-256 hash	Nunca en plaintext
Passwords	Django PBKDF2	Hasher por defecto de Django
Webhook signatures	HMAC-SHA256	Firma de payloads

Gestion de claves (OpenBao/Vault): Soporte KV v1/v2, autenticacion Token o AppRole, caching in-memory, fallback a environment variables. Paths:

`xcapit/django/config` , `xcapit/database/postgres` , `xcapit/database/redis` , `xcapit/api-keys/jwt` .

OpenBao produccion: PostgreSQL backend con SSL, HA con distributed locks, memory locking (secretos no se swapean a disco), politicas read-only para aplicaciones.

Pre-commit: `detect-private-key` bloquea commits con claves privadas.

Estado: Implementado

4. Controles Tecnologicos — Aplicacion

A.8.25 Seguridad en el ciclo de vida del desarrollo

Pregunta de auditoria: ¿Se define e implementa un SDLC seguro?

Cumplimiento:

El equipo de desarrollo sigue un conjunto de reglas de desarrollo seguro obligatorias que se aplican a todo codigo nuevo. Estas reglas se verifican automaticamente mediante linters, pre-commit hooks, y revisiones de codigo:

Reglas de desarrollo seguro mandatorias:

Regla	Detalle	Como se verifica
Type hints	Obligatorios en todas las funciones nuevas (parametros + retorno)	Mypy en CI
Import order	stdlib → Django → third-party → local	ruff (isort rules) en pre-commit
Service layer	Logica de negocio en services, NUNCA en views ni serializers	Code review obligatorio
Transaction atomicity	<code>@transaction.atomic</code> para operaciones multi-paso	Code review
Audit trail	<code>AuditService.log_from_request()</code> para operaciones significativas	Code review
UUID primary keys	En todos los modelos (previene IDOR por IDs secuenciales)	Code review
No raw SQL	Solo Django ORM. Si es unavoidable, queries parametrizadas	Code review + CodeQL
No PII en logs	Email, passwords, tokens nunca logueados	Middleware de redaccion automatica
API key hashing	Solo SHA-256 hash en base de datos, nunca plaintext	Modelo con hash en save()
Multi-tenancy	Queries siempre scoped a <code>request.user.company</code>	Tests de aislamiento automatizados

Pre-commit hooks (11 hooks enforced automaticamente):

```
# Formato y validacion
- trailing-whitespace           # Elimina whitespace al final de
    lineas
- end-of-file-fixer             # Asegura newline al final de
    archivos
- check-yaml                    # Valida sintaxis YAML
- check-json                    # Valida sintaxis JSON
- check-added-large-files       # Bloquea archivos > 1000KB
- check-merge-conflict         # Detecta marcadores de merge
    conflict sin resolver
- detect-private-key           # BLOQUEA commits con claves privadas

# Calidad de codigo
- black                         # Auto-formateo (88 chars, Python
    3.11)
- ruff --fix                    # Auto-fix linting (E/W/F/I/B/C4/UP
    rules)
- mypy                          # Type checking (on push)
- pytest                        # Suite de tests completo (on push)
```

Checklist de code review obligatorio:

Cada Pull Request debe pasar una revision que verifica 4 dimensiones:

Arquitectura: Logica de negocio en service classes (no views). Services retornan `ServiceResult[T]`. Queries scoped a `request.user.company`. `@transaction.atomic` para operaciones multi-paso. UUID PKs, TextChoices, auto timestamps.

API: Errores RFC 7807 (Problem Details). Serializadores separados para lectura y creacion. Paginacion, filtros y ordering configurados. Permissions `[IsAuthenticated, IsCompanyMember]` como minimo. `get_queryset()` filtra por empresa.

Testing: Happy path y casos de error/edge. Aislamiento multi-tenancy (`other_auth_client` obtiene 0 resultados). Acceso no autenticado retorna 401. Coverage > 90%.

Seguridad: No PII en logs. No secrets en codigo. Input validado en todos los endpoints. Rate limiting en endpoints publicos/sensibles. Errores no exponen internals. No raw SQL.

Estado: Implementado

A.8.26 Requisitos de seguridad de las aplicaciones

Pregunta de auditoria: ¿Se definen los requisitos de seguridad antes del desarrollo?

Cumplimiento:

Cada nueva funcionalidad debe cumplir las siguientes reglas de seguridad mandatorias antes de ser aceptada en el codebase. Estas reglas son verificadas durante el code review y por las herramientas automatizadas del CI:

1. **NUNCA** exponer detalles internos de error en respuestas API — stack traces, nombres de modulos, rutas del filesystem se reemplazan por mensajes genericos RFC 7807.

2. **NUNCA** almacenar API keys en plaintext — solo el hash SHA-256 se persiste.
3. **NUNCA** loguear PII (emails, passwords, tokens) — el middleware de logging redacta automaticamente estos campos.
4. **SIEMPRE** scope queries a `request.user.company` — ningun endpoint debe retornar datos cross-tenant.
5. **SIEMPRE** validar permisos antes de acceder a datos — cada ViewSet declara sus `permission_classes`.
6. **SIEMPRE** usar `@transaction.atomic` para operaciones que modifican multiples registros.
7. **SIEMPRE** registrar cambios significativos con AuditService para trazabilidad.

Análisis de amenazas (OWASP Top 10:2025): Las 10 categorías OWASP estan mapeadas a controles especificos con trazabilidad a historias de usuario. La matriz documenta que historia de usuario mitiga que categoria OWASP.

Validacion de seguridad FHE al arranque: El nivel de seguridad FHE (128/192/256 bits) se valida al arranque del servidor con `ValueError` si el valor es invalido.

Estado: Implementado

A.8.27 Arquitectura segura de sistemas y principios de ingenieria

Pregunta de auditoria: ¿Se ha documentado la arquitectura del sistema y se ha revisado la seguridad?

Cumplimiento:

La arquitectura sigue el patron **Service Layer** donde toda la logica de negocio esta encapsulada en servicios independientes de la capa HTTP:

```
class BaseService:
    """Toda logica de negocio hereda de BaseService."""
    def __init__(self, request=None):
        self.request = request
        self.context = ServiceContext.from_request(request) if
        request else None

    def require_user(self):
        """Lanza PermissionError si no hay usuario autenticado."""
    def require_company(self):
        """Lanza PermissionError si el usuario no pertenece a una
        empresa."""

class ServiceResult(Generic[T]):
    """Resultado estandar de toda operacion de servicio."""
    success: bool
    data: T | None
    error: str | None
    error_code: str | None

    @classmethod
    def ok(cls, data: T) -> ServiceResult[T]: ...
    @classmethod
    def fail(cls, message: str, error_code: str) -> ServiceResult[T]:
        ...
```

Este patron garantiza que: (a) la logica de negocio es testeable independientemente de HTTP, (b) los errores tienen formato consistente, (c) los hooks de auditoria se aplican a nivel de servicio.

Principios de ingenieria aplicados:

Principio	Implementacion concreta
Defense in depth	13 middleware layers independientes, cada uno abordando un aspecto de seguridad
Fail-closed	Operaciones de escritura sin contexto de consorcio = denegadas automaticamente
Least privilege	Permisos granulares por endpoint con 9 clases de autorizacion
Separation of concerns	7 Django apps + SDK + Dashboard + Contracts, cada uno con responsabilidad unica
Privacy by design	FHE — datos nunca descifrados en servidor por diseno criptografico
Immutable audit trail	AuditEvent con hash chain SHA-256 para eventos de gobernanza
Zero trust	Cada request pasa por autenticacion JWT + verificacion de permisos + rate limiting

Estado: Implementado

A.8.28 Codificación segura

Pregunta de auditoría: ¿Se aplican directrices de codificación segura?

Cumplimiento:

Validación de input (patrón de serializer con validación en capas):

```
class RegisterSerializer(serializers.Serializer):
    email = serializers.EmailField()           # Valida
        formato email
    password = serializers.CharField(write_only=True, min_length=12)
        # Min 12 chars
    password_confirm = serializers.CharField(write_only=True)

    def validate_email(self, value):
        if User.objects.filter(email=value).exists():
            raise serializers.ValidationError("Email already
            registered.")
        return value.lower()                 # Normaliza a
        minúsculas

    def validate_password(self, value):
        validate_password(value)              # 4
        validadores de Django
        return value

    def validate(self, attrs):
        if attrs["password"] != attrs["password_confirm"]:
            raise serializers.ValidationError({"password_confirm":
            "Passwords do not match."})
        return attrs
```

Prevención de inyección: Django ORM exclusivo (no `raw()` ni SQL directo), JSONSchemaValidator para campos JSON, no uso de `eval()`, `exec()` ni `subprocess` con input de usuario, template autoescaping de Django.

Security headers enforced en settings.py:

```
SECURE_BROWSER_XSS_FILTER = True
SECURE_CONTENT_TYPE_NOSNIFF = True
SECURE_HSTS_SECONDS = 31536000
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True
SECURE_SSL_REDIRECT = True
X_FRAME_OPTIONS = "DENY"
SECURE_REFERRER_POLICY = "strict-origin-when-cross-origin"
SECURE_CROSS_ORIGIN_OPENER_POLICY = "same-origin"
```

Linting automatizado (ruff): Reglas E/W (pycodestyle), F (Pyflakes), I (isort), B (bugbear), C4 (comprehensions), UP (pyupgrade). Enforced en pre-commit y CI.

Estado: Implementado

A.8.29 Pruebas de seguridad en desarrollo y aceptación

Pregunta de auditoría: ¿Se realizan pruebas SAST/DAST o de seguridad?

Cumplimiento:

Tipo	Herramienta	Frecuencia	Resultado
SAST	CodeQL (Python + JS)	Push, PR, semanal	Queries security-extended
Dependency scan	pip-audit + safety	Cada push	CVEs conocidas
Secret scan	TruffleHog v3.88.0	Cada push	Credentials en commits
Container scan	Trivy (CRITICAL/HIGH, bloquea build)	Cada push	SARIF → GitHub Security
Container scan	Grype (secundario)	Cada push	SARIF → GitHub Security
Unit tests	pytest	Cada push	1,496 Django + 620 SDK
Coverage	pytest-cov	Cada push	96.23% (umbral CI: 90%)
Multi-version	Python 3.9/3.10/3.11/3.12 matrix	Cada push	Compatibilidad
Smart contracts	Foundry forge test	Cada push	Todos los contratos
Frontend	ESLint + Vite build	Cada push	Sin errores

Tests de seguridad específicos incluidos en el suite: - Aislamiento multi-tenancy (Company A no ve datos de Company B) - Acceso no autenticado retorna 401 - Password debil rechazado por validadores - Email duplicado rechazado - Rate limiting enforcement verificado - JSON schema validation

Estado: Implementado

A.8.31 Separacion de los entornos de desarrollo, prueba y produccion

Pregunta de auditoria: ¿Estan separados los entornos de desarrollo, prueba y produccion?

Cumplimiento:

Los tres entornos estan completamente separados con configuraciones diferenciadas que aseguran que los datos y credenciales de produccion nunca se usan en desarrollo o testing:

Aspecto	Desarrollo	Testing	Produccion
Docker profile	dev	test	production
Database	PostgreSQL (port expuesto, password local)	SQLite in-memory	PostgreSQL (port NO expuesto, password env var)
Cache	Redis (port expuesto, sin password)	DummyCache (no-op)	Redis (port NO expuesto, password obligatoria)
DEBUG	True	True	False
Ports expuestos	8000, 5432, 6379, 8200	Ninguno	Ninguno
Filesystem	Read-write	N/A	Read-only
Password hasher	Django PBKDF2	MD5 (rapido para tests)	Django PBKDF2
Password min	12 chars	8 chars (rapido)	12 chars
Rate limiting	Habilitado	Deshabilitado	Habilitado
Brute-force	Habilitado	Deshabilitado	Habilitado
Secretos	.env file	Keys de test hardcodeadas	OpenBao/Vault + env vars
SSL/TLS	No	No	Obligatorio
Logging	Console	Deshabilitado	JSON estructurado

Dockerfile multi-stage:

```

Stage 1: builder      → Compila dependencias en /build (imagen temporal)
Stage 2: production  → Imagen minimal, non-root user, read-only filesystem
Stage 3: development → Extiende production, agrega herramientas de desarrollo

```

Las herramientas de desarrollo (debuggers, profilers) solo existen en la imagen de desarrollo y nunca llegan a produccion.

Estado: Implementado

A.8.32 Gestion de cambios

Pregunta de auditoria: ¿Se revisan y aprueban formalmente los cambios?

Cumplimiento:

Todo cambio de codigo sigue un flujo riguroso que incluye validacion automatica, revision humana, y proteccion de la rama principal:

Formato de commits obligatorio (Conventional Commits):

<type>(<scope>): <description>

Tipos: feat, fix, docs, style, refactor, perf, test, build, ci, chore
Scopes: sdk, sdk-ts, dashboard, contracts, api, cli, docs, fhe
Ejemplo: feat(api): add federated inference endpoint

Flujo completo de un cambio:

1. **Branch feature:** El desarrollador crea una rama desde main para cada cambio.
2. **Pre-commit (11 hooks):** Al hacer commit, se ejecutan automaticamente: formateo, linting, deteccion de secretos, y validacion de archivos.
3. **Push a CI:** Al pushear, el CI ejecuta 10+ jobs: lint, tests (con coverage $\geq 90\%$), security scan, container scan.
4. **Pull Request:** El desarrollador crea un PR con checklist obligatorio: tests pasan, linting limpio, docs actualizados, CHANGELOG actualizado, commits convencionales, descripcion explica el "por que" del cambio.
5. **Code review:** Minimo 1 maintainer debe aprobar el PR revisando las 4 dimensiones (arquitectura, API, testing, seguridad).
6. **CI gate:** Todos los jobs de CI deben pasar antes de poder hacer merge. No se puede hacer merge con checks fallidos.
7. **Merge:** A la rama main (protegida contra force push).
8. **Dual push:** Cada release se publica en GitHub + GitLab (ambos repositorios sincronizados).

Branch protection: main protegida contra force push, require PR reviews, status checks requeridos.

Estado: Implementado

A.8.33 Datos de prueba

Pregunta de auditoria: ¿Se ocultan los datos de produccion en los entornos de prueba?

Cumplimiento:

Los tests **nunca** usan datos de produccion. Toda la data de test es sintetica, generada por fixtures de pytest que crean empresas, usuarios y recursos ficticios para cada ejecucion:

Base de datos de test completamente aislada:

```
# settings_test.py - Base de datos separada de produccion
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",    # SQLite in-memory,
        # descartada post-test
    }
}
CACHES = {
    "default": {
        "BACKEND": "django.core.cache.backends.dummy.DummyCache",    #
        # Sin Redis
    }
}
```

Fixtures sinteticos con datos ficticios:

```
@pytest.fixture
def company(db):
    return Company.objects.create(
        name="Test Company", email="test@testcompany.com",
        industry="technology")

@pytest.fixture
def other_company(db):
    return Company.objects.create(
        name="Other Company", email="other@othercompany.com",
        industry="finance")

@pytest.fixture
def user(db, company):
    return User.objects.create_user(
        email="test@example.com", password="testpassword123",
        first_name="Test", last_name="User", company=company)

@pytest.fixture
def auth_client(api_client, user):
    refresh = RefreshToken.for_user(user)
    api_client.credentials(HTTP_AUTHORIZATION=f"Bearer
    {refresh.access_token}")
    return api_client

@pytest.fixture
def other_auth_client(api_client, other_user):
    """Cliente autenticado de OTRA empresa para tests de aislamiento
    multi-tenancy."""
    client = APIClient()
    refresh = RefreshToken.for_user(other_user)
    client.credentials(HTTP_AUTHORIZATION=f"Bearer
    {refresh.access_token}")
    return client
```

Test de aislamiento multi-tenancy:

```
def test_other_company_cannot_access(self, other_auth_client,
    company):
    """Verifica que la empresa B no puede ver datos de la empresa
    A."""
    MyModel.objects.create(name="Not yours", owner=company)
    response = other_auth_client.get("/api/v2/mymodel/")
    assert response.data["count"] == 0 # Company B recibe 0
    resultados
```

Clave de cifrado de test separada: settings_test.py usa una Fernet key de test hardcodeda que nunca se usa en produccion. La clave de produccion se obtiene de variables de entorno y el servidor rechaza arrancar sin ella.

Estado: Implementado

5. Resumen de Cumplimiento

Control	Tema	Estado
4.1	Alcance SGSI — Modulos, multi- tenancy, aislamiento	Implementado
6.1.2	Evaluacion de riesgos — OWASP Top 10:2025 completo	Implementado
A.5.1	Politica de seguridad — Divulgacion, SLA, guias	Implementado
A.5.2	Roles y responsabilidades — 9 permission classes, fail-closed	Implementado
A.5.7	Inteligencia de amenazas — 6 scanners, Dependabot	Implementado
A.5.9	Inventario de activos — Infraestructura, dependencias, contratos	Implementado
A.5.15	Control de acceso — JWT, API keys, axes, passwords	Implementado
A.5.19	Seguridad proveedores — Supply chain scanning, portabilidad	Implementado
A.5.22	Seguimiento proveedores — Health checks, CI dual, metricas	Implementado
A.5.23	Cloud security — Vercel headers, ECS Fargate, containers hardened	Implementado

Control	Tema	Estado
A.5.24	Gestion de incidentes — SLA 48h/5d/90d, Sentry	Implementado
A.5.25	Evaluacion de eventos — AuditLog, correlation IDs, hash chain	Implementado
A.5.30	Continuidad TIC — Health checks 3 niveles, restart, persistencia	Implementado
A.5.31	Requisitos legales — GDPR/HIPAA via FHE, AGPL-3.0	Implementado
A.5.33	Security review — 96.23% coverage, 2116 tests, 6 scanners	Implementado
A.5.34	Privacidad (DCP) — FHE zero-knowledge, PII redaction	Implementado
A.8.2	Privilegios de acceso — Roles, tiers, API key tracking	Implementado
A.8.8	Vulnerabilidades tecnicas — Pipeline CI automatizado, 6 scanners	Implementado
A.8.9	Gestion configuracion — Validacion al arranque, hardening, headers	Implementado
A.8.13	Copias de seguridad — WAL, AOF, volumes nombrados	Implementado
A.8.15	Registros de eventos — JSON structured, 13 middleware, correlation IDs	Implementado

Control	Tema	Estado
A.8.16	Seguimiento actividades — Sentry real-time, webhooks, usage tracking	Implementado
A.8.20	Seguridad de redes — CORS whitelist, SSRF protection, isolation	Implementado
A.8.24	Criptografia — FHE CKKS, Fernet, SHA-256, HMAC, OpenBao/Vault	Implementado
A.8.25	SDLC seguro — 10 reglas mandatorias, 11 pre-commit hooks, review	Implementado
A.8.26	Requisitos seguridad — 7 reglas absolutas, OWASP mapping	Implementado
A.8.27	Arquitectura segura — Service layer, 7 principios de ingeniería	Implementado
A.8.28	Codificación segura — Serializer validation, no SQL raw, headers	Implementado
A.8.29	Pruebas de seguridad — SAST/DAST, 2116 tests, container scan	Implementado
A.8.31	Separación entornos — 3 perfiles, configs diferenciadas	Implementado
A.8.32	Gestión de cambios — Conventional Commits, PR review, CI gate	Implementado
A.8.33	Datos de prueba — Fixtures sintéticos,	Implementado

Control	Tema	Estado
	DB aislada, key separada	

32/32 controles implementados.